Why This Is the Best-Case Solution

A detailed explanation of the graph-colouring timetable, the DSatur heuristic, and the exact branch-and-bound solver

First, an important distinction: a “best-case solution” can mean either the best possible timetable (minimum slots) or the fastest input case for the algorithms. This report explains both. The project does not promise that DSatur is always optimal. It provides a valid schedule quickly, then exact search either proves the minimum or reports that its time limit prevented proof.

1. The scheduling problem as graph colouring

Represent every course as a vertex. Draw an undirected edge between two courses when at least one student takes both. A time slot is a colour. Adjacent vertices must have different colours. The additional hall limit means no colour may be used by more than **R** courses. The task is therefore a capacitated graph-colouring problem.

The output is not only a colour count. It is an assignment of every course to exactly one slot, such that every conflict and capacity rule holds. The timetable validator checks those rules before the UI treats a result as valid.

2. What “best possible” means

For any valid schedule, two simple lower bounds apply:

Capacity bound: at least ceil(N / R) slots are required, because each slot holds at most R courses.
Conflict bound: if a set of courses are all pairwise in conflict (a clique of size W), at least W slots are required.

Thus the true optimum is at least max(ceil(N/R), W). The current exact solver uses the capacity lower bound directly for pruning. Conflict restrictions are enforced during assignments, but this implementation does not compute a maximum-clique bound as a separate optimization. This distinction matters when describing what the code does.

A particularly clear proof of optimality occurs when the schedule uses exactly ceil(N/R) slots and there are no stronger conflict constraints requiring more. No valid schedule can use fewer slots, so matching this lower bound proves the timetable optimal. More generally, any result is proven only after the exact search has ruled out every schedule with fewer slots, or a valid lower bound meets its slot count.

3. Why DSatur is a strong fast heuristic

DSatur repeatedly selects the unscheduled course whose already-scheduled neighbours use the largest number of distinct slots. That quantity is its saturation degree. It then assigns the course to the first slot that has room and contains none of its conflicting neighbours. If no existing slot works, DSatur opens another.

Choice	Why it helps
Highest saturation first	A course with many differently coloured neighbours has fewer safe choices. Scheduling it early reduces the chance that it is trapped late.
Degree tie break	Among equally saturated courses, prioritize the one with more conflicts. It is likely to constrain more future decisions.

Choice	Why it helps
Lowest feasible slot	Reusing an available slot keeps the colour count low; the capacity check also respects the hall limit.
Deterministic final tie break	Course id breaks remaining ties, making a run reproducible for a fixed graph.

DSatur is called a heuristic because its local choices do not consider every possible future assignment. It always returns a valid result for a valid graph and positive capacity, but another sequence of choices may use fewer slots. Its result is an upper bound: a known valid schedule that the exact solver can try to improve.

4. Why exact branch-and-bound can prove the minimum

The exact solver starts with the DSatur schedule as its incumbent, or current best valid solution. It builds other assignments recursively. At each step it chooses a high-saturation unassigned course, tests slots that satisfy both constraints, and backtracks when a partial assignment cannot lead to a better result.

Technique	Effect
Upper bound from DSatur	Search only needs to find a schedule with fewer slots than the existing valid schedule.
Capacity bound	If the remaining lower bound is already as large as the incumbent, that branch cannot improve the answer and is pruned.
Symmetry breaking	New slots are introduced in order. Permuting labels such as “slot 1” and “slot 2” does not create a genuinely different schedule, so equivalent branches are avoided.
Conflict and hall checks	A course is placed only if no conflicting course is in that slot and the slot has fewer than R courses. Invalid partial assignments are never expanded.
Time limit status	If interrupted, return the best valid schedule found and mark optimality unproven. Completion means the search has ruled out every improvement and proves optimality.

“Exact” describes the method, not every possible run outcome. A finished exhaustive branch-and-bound search proves the optimum. A timed-out run remains a best-known valid answer, not a proof.

5. Worked example: 8 courses, capacity 3

Suppose there are eight courses and each slot can host three exams. The capacity lower bound is ceil(8/3) = 3 slots. Consider this valid assignment:

```
Slot 1: Course 1, Course 4, Course 7
Slot 2: Course 2, Course 5, Course 8
Slot 3: Course 3, Course 6
```

Every slot contains at most three courses. If no pair listed together is a conflict, this is a valid schedule. It uses three slots, exactly the capacity lower bound; therefore no schedule can use two slots, and this result is optimal. The exact solver can prove this immediately from the lower bound once it sees that the incumbent already uses three slots. DSatur may also find it, but DSatur alone does not provide the proof.

If a conflict exists within a listed row, that assignment is invalid even if capacity is satisfied. The graph constraint and capacity constraint must both hold. A valid example is graph-dependent: the same slot grouping cannot be claimed for arbitrary edge lists.

6. Best-case runtime vs worst-case runtime

There is no single runtime for all inputs of a given size. Structure matters. The best case for exact search occurs when the initial DSatur timetable already meets a strong lower bound, so the solver can prove optimality without exploring a large search tree. If it does not, branch-and-bound may need to explore many assignments. Graph colouring is computationally hard in general, and the exact search can grow exponentially with the number of courses.

Part	Best-case behavior	Worst-case note
DSatur	Makes a straightforward greedy pass and returns a valid timetable. In this implementation it repeatedly rescans unassigned vertices and their neighbours.	The rescans cost roughly O(N(N+M)) with the current simple data structures; bookkeeping variants can improve this.
Exact search	A lower bound matching the DSatur incumbent can certify optimality near the root.	Backtracking may examine exponentially many partial colourings; timeout exists for responsiveness.
Graph creation / display	Separate from solver measurement.	Rendering and CSV writing are not algorithm runtime.

The UI measures solver time separately from drawing the graph and exporting CSV. Benchmarks compare both algorithms on the same generated graph for each N. An exact data point marked timed out must not be interpreted as proven optimal.

7. How to interpret the application results

When both algorithms show the same number of slots and exact search says “proven optimal,” then the DSatur timetable is also optimal for that particular graph: it matches a proven minimum. If DSatur uses more slots, it remains valid, while exact search has demonstrated a better minimum if its result is proven. If exact search times out, compare slot counts as incumbent results only; optimality is still unknown.

Displayed result	Correct interpretation
DSatur: valid; optimality not guaranteed	A feasible heuristic timetable. Its slot count is an upper bound on the minimum.
Exact: valid; proven optimal	The search completed and proved that no timetable with fewer slots exists.
Exact: valid; optimality unproven (time limit)	A best-known timetable at interruption. The true minimum may be smaller.
Both use K; exact proves K optimal	The greedy result is optimal for this input too, because it also uses K slots.

8. CSV output and visual evidence

The CSV uses one row per slot and one column per hall position. Every course should appear once. Empty cells mean a hall position is unused. In the graph view, the node colour is the assigned slot colour; the timetable grid uses the same colour for courses in that slot. These views make it easier to inspect the returned schedule, but the shared validator is the authority for checking validity.

9. Examples of best-case graph structures

The fastest proofs often happen when simple graph structure makes a lower bound decisive. These examples show why the same algorithm can finish almost immediately on one graph and search much longer on another graph with the same N.

Graph and capacity	Minimum slots	Reason
No conflicts; N=8, R=3	3	Capacity requires ceil(8/3)=3. DSatur packs isolated courses into three slots; exact can prove the lower bound is met.
No conflicts; N=8, R=8	1	All courses fit in one slot, and at least one slot is needed when N is positive.
Every pair conflicts; N=5	5	The five courses form a clique. Each needs its own slot, regardless of hall capacity.
Some conflicts; capacity bound is weak	May exceed ceil(N/R)	Conflict constraints may force more slots than hall capacity alone. Exact search must establish whether fewer slots are possible.

10. Summary: why this approach is appropriate

The combination is useful for an academic demonstration because the two methods make different trade-offs visible. DSatur gives a quick, understandable schedule. Branch-and-bound starts from that schedule and searches systematically for improvements, using a lower bound and symmetry breaking to reduce work. The timetable is accepted only if it satisfies the conflict and hall limits; the app states whether optimality is guaranteed or still unknown.

Takeaway: the best solution is the valid timetable with the fewest slots. The best-case proof is clearest when the schedule reaches a mathematical lower bound. DSatur helps find a good timetable quickly; completed exact search supplies the proof. Neither a low runtime nor a matching heuristic count by itself proves optimality.

Project reference: Exam Timetable Scheduling Using Graph Colouring, Java 17 / JavaFX implementation. The exact solver in the project uses DSatur ordering, capacity-based pruning, slot symmetry breaking, and a user-configurable time limit.